

Product Requirements Document: Polyglot Language Learner

Overview

Polyglot Language Learner is a browser-based web application for people learning and maintaining proficiency in multiple foreign languages at once. The product combines document upload, free-form text input, structured language-learning prompts, OCR, and multiple LLM backends into one guided study workspace built for multilingual learners rather than only native-language-to-target-language workflows.[cite:15][cite:1][cite:3]

The central differentiator is bridge-language learning. The app should let a user study one target language through another already acquired language instead of assuming English is always the explanation language. A unified LLM gateway architecture is a good fit because it provides one integration surface for model selection, routing, fallback handling, and observability across multiple providers.[cite:1][cite:2][cite:3]

Product vision

The product should help multilingual learners build new proficiency while strengthening languages they already know. A user should be able to learn French through Spanish, Thai through Indonesian, or Japanese through Thai, using explanation and study outputs that reinforce multiple languages in the same workflow.

The app should function as a document-to-study and text-to-study workspace rather than a generic chatbot. OCR and document extraction tools demonstrate the value of converting PDFs and images into usable text, and this product extends that pattern into a language-learning environment with guided tasks, bridge-language support, editable outputs, and exportable study artifacts.[cite:15]

Product positioning

Polyglot Language Learner is designed for multilingual self-directed learners, advanced students, travelers, tutors, and researchers who move between several languages and want flexible study workflows. The product is especially well suited to learners who prefer using a strong second or third language as the explanation language for a new target language.

The product also differs from conventional flashcard generation tools because it explicitly supports non-English language pairs. This includes workflows for Anki-ready deck generation where neither side of the card is English, along with optional reversible-card exports for bidirectional review.[cite:19][cite:21]

Goals

Primary goals

- Support study across multiple target languages in one application.
- Support learning a target language through a bridge language already known by the user.
- Accept both uploaded documents and direct text entry as learning inputs.
- Provide reusable guided prompt workflows for common study tasks.
- Route tasks through a backend layer that can use multiple LLMs behind one interface.
[cite:1][cite:2]
- Produce editable outputs that can be exported into tools such as Anki.[cite:21]

Success criteria

- A user can start from either an uploaded PDF or image, or from manually entered text.
- A user can set a user language that differs from their native language and from the source language.
- The product can generate useful study outputs for non-English language pairs.
- The product offers structured prompt templates so users do not need to invent complex prompts.
- The backend can switch models without requiring frontend workflow changes.[cite:1]
[cite:3]

Target users

Primary users

- Polyglots maintaining and expanding several foreign languages simultaneously.
- Language learners who prefer to study through a strong second or third language.
- Anki users who want high-quality deck content from text, images, and documents.[cite:21]
- Advanced self-directed learners who want roots, morphology, grammar, monolingual definitions, synonyms, and example sentences.
- Teachers, tutors, and independent researchers working with multilingual material.

User profile assumptions

The initial design should support users who may be native in one language but prefer explanations in another acquired language. The product should be comfortable for users studying among Japanese, Indonesian, Thai, Spanish, and English and should not treat non-English explanation languages as edge cases.[cite:17]

The app should also support users who actively maintain several languages and want outputs tailored to specific language pairs rather than one universal mode. Existing user behavior

shows strong interest in root-word analysis, morphology, multilingual study, and reversible flashcard directions.[cite:19][cite:23][cite:28]

Core concept

Every study action in the app should be modeled around explicit language roles:

- Source language: the language of the input text being studied.
- User language: the language used for explanations, definitions, and guidance.
- Target output language: the language used for requested output when it differs from the user language.
- Comparison language: a future optional language used for contrast or parallel explanation.

This model allows the user to reinforce acquired languages while learning a new one. It also supports outputs such as target-language monolingual definitions followed by bridge-language explanations and non-English flashcard pairs.

Final product decisions

The following decisions are part of the final MVP definition for this PRD:

- Prebuilt presets should prioritize Thai-Indonesian, Japanese-Thai, Japanese-Indonesian, Spanish-Japanese, and all supported languages with English as a fallback option.
- The word-analysis workflow should use language-aware terminology rather than one universal label.
- Monolingual output in the target language should appear by default unless the user requests otherwise.
- Comparison-language support should be planned in the data model now but shipped in Phase 2 rather than MVP.
- Anki export should include optional reversible-card layouts by default.[cite:19][cite:21]

Scope

In scope for MVP

- Upload PDFs and image files.
- Enter free-form text manually.
- Choose from structured prompt templates for text and document workflows.
- Select source language, user language, and output preferences.
- OCR or vision-based text extraction for uploaded files.[cite:15]
- Guided workflows for translation, roots, morphology, grammar, reading support, and flashcard generation.
- Editable outputs.
- Export to plain text and CSV or TSV for Anki-ready workflows.[cite:21]

- Multi-model routing through a unified backend gateway.[cite:1][cite:3]
- Preset language-pair configurations for the prioritized pairs above.

Out of scope for MVP

- Native mobile apps.
- Full spaced repetition scheduling within the app.
- Real-time collaborative study sessions.
- Voice conversation practice.
- Broad handwriting recognition.
- Comparison-language output visible in the standard workflow.

Input modes

Mode 1: Document and image upload

The app must support PDFs and image files that contain language material from books, signs, websites, screenshots, worksheets, and photographed pages. OCR and document extraction workflows show that these sources can be converted into structured text for downstream analysis and study use.[cite:15]

Requirements:

- Accept PDF, JPG, JPEG, PNG, and optionally HEIC through conversion.
- Support multi-page processing.
- Support page preview, page selection, rotation, and zoom.
- Store extracted text by page.
- Allow OCR reruns independent of downstream tasks.
- Preserve page references in final outputs.

Mode 2: Free-form text input

The initial module must also include a free-form text box so users can paste or type language content directly. This supports quick dictionary-style exploration, grammar questions, root analysis, morphology analysis, and flashcard generation without requiring a file upload.

Requirements:

- Large multilingual text input field.
- Prompt-template picker integrated into the text-input flow.
- Character count and simple validation.
- Support single-word, phrase, sentence, paragraph, and longer text inputs.
- Preserve the current language-setting context across intake modes.

Guided prompt system

The product should offer structured prompt templates instead of requiring the user to invent prompts from scratch. Templates should work with both uploaded content and manually entered text, with an advanced custom-prompt mode available but not central to the MVP workflow.

Requirements:

- Prompt templates appear as cards, menu items, or a structured selector.
- Each template must explain what the user will receive.
- Templates must map to reusable backend prompt logic.
- Users may modify a chosen template in advanced mode.
- Templates must explicitly incorporate source language and user language.

Initial prompt templates

The first release should include at least the following:

1. Extract text exactly.
2. Translate naturally.
3. Explain sentence by sentence.
4. Build a vocabulary list.
5. Explain grammar patterns.
6. Add reading support or transliteration.
7. Create flashcards.
8. Answer questions about this text.
9. Analyze a root word, base form, stem, or morphology depending on language.

Word-analysis workflow

One of the core text-input templates should be a structured word-analysis workflow. The user-facing label should vary by language so the terminology remains accurate and intuitive.

Language-aware naming strategy

- Indonesian: “Root word” is the preferred default label because it aligns with common learner expectations and productive derivational analysis.[cite:23][cite:28]
- Thai: use a more neutral label such as “Base form” or “Word analysis” until a stronger language-specific convention is chosen.
- Japanese kana or inflected forms: use “Morphology.”
- Japanese multi-kanji compounds: use “Kanji breakdown + morphology.”
- Generic fallback in mixed-language UI contexts: “Word analysis.”

Word-analysis output structure

Expected output should include:

- Root word, base form, stem, or morphology analysis as appropriate.
- Derivational explanation where applicable.
- Other related words built from the same root or pattern.
- Monolingual definition in the target or source language being studied.
- Source-language or target-language synonyms where appropriate.
- Example sentences in the studied language.
- User-language explanation.
- Optional user-language synonyms and example sentences.
- Notes about register, nuance, kanji components, affixes, or grammar where relevant.

Japanese-specific behavior

For Japanese, the workflow should not only handle inflection or morphological analysis for kana-based words, but also break down the specific kanji and their meanings when the word consists of two or more kanji. This makes the feature more useful for learners who benefit from semantic decomposition in addition to ordinary dictionary glossing.

Monolingual-first output policy

Monolingual output in the target language should be shown by default unless the user requests otherwise. This supports stronger immersion and encourages learning through the studied language as a good default pedagogical practice.

Default output ordering should be:

1. Monolingual definition or explanation in the target language.
2. Example sentences in the target language.
3. User-language explanation.
4. Additional user-language support notes as needed.

Users should be able to reduce or disable monolingual depth, but the default should favor target-language-first presentation.

Multilingual bridge-language learning

Bridge-language learning is a core feature. The app must let the user study one language through another acquired language instead of assuming only native-language support.

Core requirements

- User profile must support native language plus multiple proficient languages.[cite:17]
- User language must be independently selectable from source language.
- User language must be changeable per task or per session.
- Prompt templates must explicitly use the selected user language.
- Exports must support non-English language pairs.

MVP preset pairs

The product should launch with preset configurations for:

- Thai → Indonesian
- Japanese → Thai
- Japanese → Indonesian
- Spanish → Japanese
- All supported languages → English

These presets should prefill explanation language, export direction, and recommended prompt defaults.

Flashcards and Anki workflows

The app should be built from the start with Anki-compatible export workflows. Existing user behavior shows that reversible card generation matters in practice, and this is even more important when neither side of a card is English.[cite:19][cite:21]

Requirements

- Export flashcard rows in CSV or TSV suitable for Anki import.
- Support user-defined front and back language directions.
- Include optional reversible two-way layouts by default.
- Support tags for language pair, topic, level, and source text.
- Preserve UTF-8 text for all scripts.
- Support non-English-to-non-English card pairs.
- Remember the last-used export mode per language pair in a future enhancement.

Example deck directions

- Thai front, Indonesian back.
- Japanese front, Thai back.
- Japanese sentence front, Indonesian explanation back.
- Spanish front, Japanese back.

- Any supported language front, English back.

Functional requirements

Module 1: Unified intake screen

Requirements:

- Two clear tabs or panels: Upload file and Enter text.
- Shared language-setting context across both modes.
- Prompt-template picker available in both modes.
- Quick-start examples.
- Mobile-friendly layout.

Module 2: Upload and OCR workflow

Requirements:

- Accept supported file types.
- Validate type, size, and page count.
- Show thumbnails and page selection.
- Extract text with OCR or a vision workflow.[cite:15]
- Allow OCR correction before downstream prompting.
- Preserve page references.

Module 3: Manual text workflow

Requirements:

- Support word, phrase, sentence, paragraph, or longer input.
- Support immediate use of structured templates.
- Suggest detected language with user override.
- Allow quick rerun with another user language or prompt.

Module 4: Language settings

Requirements:

- Source language selector or auto-detect.
- User language selector.
- Learner level selector.
- Output style preferences such as concise, detailed, literal, or natural.
- Session memory of current language context.

- Data-model support for future comparison language.

Module 5: Prompt orchestration

Requirements:

- Each template maps to structured backend instructions.
- Prompts explicitly include source language and user language.
- Prompts request labeled outputs for predictable rendering.
- Advanced custom prompt editing is supported but secondary.

Module 6: Multi-model backend

A backend proxy with a unified gateway remains the preferred architecture because it centralizes keys, routing, fallback handling, and observability across models and providers.
[cite:1][cite:2][cite:3]

Requirements:

- Frontend never exposes provider keys.[cite:2][cite:12]
- Backend supports configurable model defaults by task type.
- Backend supports fallback or alternate routing in later versions.[cite:1][cite:2]
- Backend records latency, usage, and error metadata.[cite:2][cite:12]

Module 7: Results workspace

Requirements:

- Structured panels for source text, generated output, and optional notes.
- Clear language labels on each section.
- Editable outputs.
- One-click copy by section.
- Export actions for notes, vocabulary, and flashcards.
- Ability to regenerate with another user language or model.

UX requirements

Product principles

- Guide users with task-based workflows.
- Treat multilingual users as the primary audience.
- Keep language roles explicit at all times.
- Make outputs easy to reuse externally.

- Favor target-language-first learning by default.

Key screens

Screen 1: Unified intake

Must include:

- Product name: Polyglot Language Learner.
- Multilingual value proposition.
- Upload tab.
- Enter-text tab.
- Prompt-template picker.
- Source language and user language settings.

Screen 2: Preview or text confirmation

Must include:

- For uploads, page thumbnails and selected pages.
- For text input, entered text review and language confirmation.
- Continue action into task processing.

Screen 3: Task and prompt setup

Must include:

- Selected template summary.
- Language settings.
- Learner level.
- Model selection.
- Advanced prompt edit option.

Screen 4: Processing state

Must include:

- Intake status.
- OCR status when relevant.
- Generation status.
- Clear retry messaging.

Screen 5: Results workspace

Must include:

- Source panel.
- Output panel with language labels.
- Edit and rerun controls.
- Export options.
- Flashcard preview where applicable.

Data model

UserProfile

Fields:

- `userId`
- `nativeLanguages`
- `proficientLanguages`
- `studyLanguages`
- `preferredUserLanguage`
- `preferredExportFormats`
- `flashcardPreferences`

StudyContext

Fields:

- `contextId`
- `sourceLanguage`
- `userLanguage`
- `targetOutputLanguage`
- `comparisonLanguage`
- `learnerLevel`
- `outputStyle`
- `selectedModel`

Document

Fields:

- documentId
- originalFilename
- mimeType
- fileSize
- pageCount
- uploadTimestamp
- processingStatus

Page

Fields:

- pageId
- documentId
- pageNumber
- previewImageUrl
- rotation
- ocrText
- ocrConfidenceSummary
- selected

TextInputSession

Fields:

- textInputId
- rawInputText
- detectedLanguage
- sourceLanguageConfirmed
- createdAt

TaskRun

Fields:

- taskRunId
- inputType
- documentIdOrTextInputId

- taskType
- sourceLanguage
- userLanguage
- targetOutputLanguage
- comparisonLanguage
- learnerLevel
- modelId
- promptTemplateId
- runStatus
- createdAt
- completedAt

StudyItem

Fields:

- studyItemId
- taskRunId
- sourceText
- reading
- rootOrBaseForm
- kanjiBreakdown
- monolingualDefinition
- userLanguageDefinition
- sourceSynonyms
- userLanguageSynonyms
- sourceExamples
- userLanguageExamples
- partOfSpeech
- sourcePage
- tags

Non-functional requirements

Security

Gateway and proxy patterns are important because they centralize authentication and access control while keeping provider credentials out of the browser.[cite:2][cite:12]

Requirements:

- No direct provider API key exposure.
- Validated uploads.
- Sanitized logs.
- Clear retention and deletion controls.

Privacy

Requirements:

- Explain whether uploads and text inputs are stored.
- Allow deletion of stored source material and outputs.
- Separate analytics from user content where practical.

Reliability

Requirements:

- Distinguish upload, OCR, and generation failures.
- Provide model-switch retry options where useful.
- Preserve settings when rerunning.

Accessibility

Requirements:

- Keyboard support.
- Screen-readable labels.
- Sufficient contrast.
- Mobile-responsive controls.

API requirements

Suggested internal endpoints:

- POST /api/uploads
- POST /api/text-inputs
- POST /api/tasks/run
- POST /api/tasks/{id}/rerun

- GET /api/tasks/{id}/status
- GET /api/exports/{id}
- DELETE /api/documents/{id}
- DELETE /api/text-inputs/{id}

The backend should continue using a provider-neutral request pattern where model choice is a request parameter rather than hard-coded logic. OpenAI-compatible gateway designs show why this approach is practical for products that need to work across multiple downstream models.[cite:1][cite:3]

Analytics and telemetry

The backend should capture operational events that help improve template quality, model defaults, and export workflows. AI gateway systems emphasize observability as a production concern, and the same principle applies here.[cite:2][cite:12]

Recommended events:

- Input mode selected.
- Upload started and completed.
- Text input submitted.
- Prompt template selected.
- User language changed.
- OCR completed.
- Task completed.
- Export generated.
- Flashcard export generated.
- Error category triggered.

Recommended metrics:

- Task completion rate by template.
- Export rate by template.
- Rerun rate by user language.
- OCR success rate.
- Average latency by task and model.
- Share of non-English-to-non-English exports.

Release plan

Phase 1: Final MVP

Deliver:

- Polyglot branding and multilingual positioning.
- Unified intake screen with upload and manual text tabs.
- Prompt-template system.
- Language-aware word-analysis workflow.
- Source language and user language controls.
- Monolingual-first output presentation.
- OCR extraction for uploaded documents.[cite:15]
- Results workspace with editable output.
- Anki-friendly exports including reversible layouts and non-English pairs.[cite:19][cite:21]
- Backend proxy with unified LLM gateway support.[cite:1][cite:3]
- Preset language-pair configurations.

Phase 2: Comparison-language and power features

Deliver:

- Comparison-language support.
- Saved language-pair presets.
- Side-by-side multilingual explanation modes.
- Better morphology and derivation workflows.
- Batch flashcard generation and deck packaging.

Phase 3: Advanced study platform

Deliver:

- Saved multilingual projects.
- Cross-language review dashboards.
- Teacher and tutor workflows.
- Deeper analytics by language pair and task type.

Acceptance criteria for MVP

The MVP is acceptable when all of the following are true:

- The intake screen supports both file upload and free-form text input.
- The product supports at least one language-aware word-analysis workflow.
- The user can choose a user language different from the source language.
- The product can generate outputs for a non-English language pair.
- Uploaded documents can be converted into editable text through OCR.[cite:15]
- The backend can route tasks through a unified gateway and return selected-model results. [cite:1][cite:3]
- The user can export flashcard-ready output with optional reversible layouts for Anki. [cite:19][cite:21]

Risks and mitigations

Risk	Impact	Mitigation
Users become confused by source language versus user language	Lower task success	Use explicit labels, examples, and presets
Root or morphology analysis is unreliable in some languages	Lower trust	Use language-aware naming and confidence-sensitive wording
OCR errors contaminate study output	Poor explanations and cards	Allow OCR correction and rerun workflows[cite:15]
One model performs poorly for a given language pair	Uneven quality	Support task-based model defaults and model switching[cite:1][cite:2]
Non-English exports break in external tools	Export friction	Preserve UTF-8 and test Anki import templates[cite:21]

Recommended implementation approach

The recommended architecture remains a responsive web application backed by a lightweight server-side API. The browser should handle upload, preview, manual text input, editing, and export UI, while the backend handles OCR orchestration, prompt-template expansion, model routing, secure provider access, logging, and usage monitoring.[cite:2][cite:12][cite:15]

This architecture best supports the product’s core promise: one workspace for multilingual study, one set of structured learning workflows, and flexible language-pair outputs that reinforce acquired languages while building new ones.[cite:1][cite:2][cite:3]